1. INTRODUCTION

1.1PROJECT DESCRIPTION:

The modern banking sector relies heavily on data-driven decision-making to optimize operations and reduce financial risks. Among various banking operations, **loan approval** is one of the most critical tasks, as it directly impacts the bank's financial stability and customer satisfaction. Traditionally, banks rely on manual verification and conventional scoring systems to approve or reject loan applications. While these methods are effective to an extent, they are often time-consuming, prone to human error, and inconsistent in decision-making across different branches or officers.

The goal of this project is to develop a machine learning model for predicting bank loan approvals. By leveraging historical loan application data, the system can accurately predict whether a loan application is likely to be approved or rejected. This prediction system aims to complement or potentially replace traditional supervised classification models used in banking, offering high accuracy, efficiency, and consistent decision-making.

The system is implemented as a web-based application, where users can input loan application details, and the model returns the predicted approval status. The interface is designed to be user-friendly, allowing bank officers or customers to easily check the loan status without complex procedures.

Background of the Project:

The banking industry plays a vital role in economic development by providing financial assistance to individuals and businesses. One of the core services offered by banks is loan approval**,** which involves evaluating a customer’s eligibility based on financial and personal details. Loan approval decisions must be accurate because incorrect approvals can lead to financial losses, while incorrect rejections may result in loss of potential customers and reduced customer satisfaction.

Traditionally, banks use manual verification processes and rule-based credit scoring systems to assess loan applications. Bank officers examine documents such as income statements, credit history, employment records, and repayment capacity before making a decision. Although this system has been followed for many years, it has certain limitations. Manual evaluation is time-consuming, requires significant human effort, and may vary

depending on the experience or judgment of the officer. Moreover, traditional systems may not effectively analyze large volumes of data or identify hidden patterns within customer information.

With the advancement of technology and the growth of digital banking, financial institutions now generate and store vast amounts of customer data. This creates an opportunity to use data analytics and machine learning techniques to improve decision-making processes. Machine learning algorithms can analyze historical loan data, learn patterns from past approvals and rejections, and make predictions for new applications. These models help reduce human bias, improve consistency, and increase prediction accuracy.

In recent years, many financial organizations worldwide have adopted machine learning-based credit assessment systems to enhance operational efficiency and reduce financial risks. Automated loan prediction systems support faster processing, minimize paperwork, and provide real-time results. They also help banks manage risk by identifying high-risk applicants before approval.

This project is developed based on this technological advancement in banking. It aims to design and implement a machine learning-based loan approval prediction system using historical loan data. The system analyzes various applicant features such as income, credit history, employment status, and loan amount to predict whether the loan will be approved or rejected.

The proposed system is implemented as a web-based application to ensure ease of access and usability. It provides a simple interface where users can enter loan details and instantly receive a prediction result. By integrating machine learning with a web application, the project demonstrates how intelligent systems can support modern banking operations.

Overall, this project reflects the growing importance of data-driven solutions in the financial sector and highlights how machine learning can enhance accuracy, efficiency, and reliability in loan approval decision-making.

Objectives of the Project

The primary objective of the project is to develop a machine learning-based bank loan approval prediction system that can classify loan applications into approved or rejected categories. The system aims to achieve the following objectives:

1. **Accurate Predictions:** Utilize supervised machine learning algorithms to maximize prediction accuracy. The model will compare multiple classification algorithms to determine the best-performing model.
2. **Time Efficiency:** Reduce the time and effort required by bank officers to manually review loan applications.
3. **Automation:** Automate the decision-making process to minimize human intervention and errors.
4. **User-Friendly Interface:** Provide an intuitive web-based platform using Flask, allowing both bank staff and customers to interact with the system efficiently.
5. **Data-Driven Decisions:** Leverage historical datasets to ensure that loan approval decisions are consistent, transparent, and data-backed.

Through these objectives, the system intends to provide banks with a reliable, accurate, and efficient tool for loan management, ultimately improving customer satisfaction and operational efficiency.

Necessity of the Project:

The necessity of a machine learning-based loan approval system arises from the limitations of traditional manual methods. In conventional banking practices, loan approval involves multiple steps, including document verification, credit scoring, and officer judgment. These methods have several drawbacks:

- **Time-Consuming:** Manual processing can take several hours or even days, especially for high-volume applications.
- **Inconsistency:** Different officers may evaluate applications differently, leading to inconsistent approval decisions.
- **Human Error:** Manual calculations and evaluations are prone to mistakes, which may result in financial losses.
- **Inefficiency:** High workloads during peak periods can slow down processing and negatively affect customer experience.

This project addresses these challenges by automating the decision-making process. The system not only reduces manual effort but also ensures accuracy and consistency, enabling banks to handle large volumes of applications efficiently. Furthermore, customers can check the status of their loan instantly, which enhances transparency and improves trust in banking operations.

Software Development Methodology:

In software development, various methodologies can be used depending on the project requirements. Common models include:

- **Waterfall Model:** Linear sequential approach, where each phase must be completed before the next begins.
- **Iterative Model:** Focuses on repeated cycles of development and refinement.
- **Spiral Model:** Combines iterative development with risk assessment.
- **V-Model:** Verification and validation-focused model.
- **Big Bang Model:** Focused on small projects without formal planning.

For this project, the Waterfall Model was chosen due to its structured approach. The project was developed in the following stages:

1. **Requirement Analysis:** Understanding the dataset, business logic, and user requirements.
2. **System Design:** Designing the architecture, database schema, and user interface.
3. **Implementation:** Writing code for data preprocessing, machine learning model development, and Flask-based web interface.
4. **Testing:** Validating the model's accuracy, usability testing of the web application, and debugging.
5. **Deployment:** Integrating the application for end-users with secure hosting.

The waterfall model ensures systematic development, clear documentation, and easier management, making it ideal for structured projects like this one.

Overview of the Designed Project:

The Bank Loan Approval Prediction System follows a machine learning-based pipeline combined with a web interface for users. The project is divided into the following stages:

Dataset Collection

The system uses historical loan application data, which includes attributes such as:

- Applicant income
- Co-applicant income
- Credit history
- Loan amount
- Loan term
- Property area
- Gender, marital status, dependents

This dataset is collected from verified banking records or publicly available repositories.

Data Preprocessing

Before training the machine learning model, the dataset undergoes several preprocessing steps:

- **Handling Missing Values:** Missing or null values are imputed using median or mode values.
- **Encoding Categorical Variables:** Non-numerical features like gender or property area are encoded using one-hot or label encoding.
- **Normalization/Scaling:** Features like income or loan amount are normalized to ensure uniformity.

Data Visualization

Visualization techniques, such as histograms, box plots, and correlation heatmaps, are applied to:

- Understand feature distributions
- Identify relationships between attributes
- Detect outliers that could skew model performance

Machine Learning Algorithm Implementation:

Multiple supervised learning algorithms are applied to the preprocessed dataset to determine the best predictive model:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- Gradient Boosting Classifier
- Support Vector Machines (SVM)

The system evaluates the models based on accuracy, precision, recall, and F1-score. The best-performing model is then serialized using Pickle for deployment.

2. SYSTEM STUDY

2.1 EXISTING SYSTEM:

Anomaly detection works by profiling individual behavior and identifying deviations from normal patterns. In online banking fraud detection, it faces several limitations. First, historical behavioral data for a single user is often insufficient to build an accurate profile. Second, the heterogeneous nature of transaction data makes it difficult to handle different types of attributes uniformly, which complicates model development. Third, transaction data is highly skewed, making it challenging to use label information effectively. These issues often lead to poor generalization and a high false alarm rate. To address this, the anomaly detection problem can be reformulated as a pseudo-recommender system problem using embedding-based methods, which leverages information from similar users through collaborative filtering and provides concise representations for further analysis.

Disadvantages:

1. The approach relies on mathematical modeling and does not employ machine learning algorithms.
2. Class imbalance was not addressed, and appropriate measures were not implemented.

2.2 PROPOSED SYSTEM:

Exploratory data analysis of loan approval involves combining multiple datasets from different sources to create a generalized dataset. Various machine learning algorithms are then applied to extract patterns and achieve maximum prediction accuracy.

In the data wrangling stage, the dataset is loaded, checked for cleanliness, and trimmed to remove inconsistencies or irrelevant information. Each cleaning decision is carefully documented and justified.

The collected dataset is split into training and test sets, typically in a 70:30 ratio. Machine learning models are trained on the training set and evaluated on the test set to measure prediction accuracy.

The classification model for loan approval is effective because it handles outliers, irrelevant variables, and a mix of continuous, categorical, and discrete features. It also provides reliable error estimates and is relatively easy to tune.

The advantages of this approach include:

1. The ability to evaluate and compare algorithm performance and
2. To address class imbalance using machine learning techniques.

3. SYSTEM SPECIFICATION

3.1 HARDWARE REQUIREMENT:

- Processor : Intel core i3
- RAM : 8GB
- HardDisk : 500GB

3.2 SOFTWARE REQUIREMENT:

- OS : Windows
- Framework : Flask
- Technology : Machine Learning using Python
- Web Browser : Chrome, Microsoft Edge
- Code editor : VisualStudio Code, Google Colab, Anaconda or Jupyter notebook.

3.3 ABOUT SOFTWARE:

PYTHON

Python is an interpreter, object-oriented, high-level programming language with dynamic semantics. Its high-level built in data structures, combined with dynamic typing and dynamic binding; make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Python supports modules and packages

Syntax

The” hello world” program is written in python

print ("Hello, Python!");

Python is a very simple language, and has a very straightforward syntax. It encourages programmers to program without boilerplate (prepared) code. The simplest directive in Python is the "print" directive - it simply prints out a line

Data types

Variables can store data of different types, and different types can do different things.Python has the following data type built-in by default, in these categories:

- Text Type : Str
- Numeric Types : int, float, complex
- Sequence Types: list, tuple, range
- Mapping Type : Dict
- Set Types : set, frozenset
- Boolean Type : Bool

- Binary Types : bytes, bytearray, memorview

Function

A function is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result. A function is a set of statements that take inputs, do some specific computation and produces output. The idea is to put some commonly or repeatedly done task together and make a function, so that instead of writing the same code again and again for different inputs, we can call the function .Python provides built-in functions like print(), etc. but we can also create your own functions. These functions are called user-defined functions.

Object

Python is an object oriented programming language. Unlike procedure oriented programming, where the main emphasis is on functions, object oriented programming stress on objects. Object is simply a collection of data (variables) and methods (functions) that act on those data. And, class is a blueprint for the object.

Features

- Easy to Learn and Use. Python is easy to learn and use.
- Expressive Language. Python language is more expressive means that it is more understandable and readable.
- Interpreted Language.
- Cross-platform Language.
- Free and Open Source.
- Object-Oriented Language.
- Extensible.
- Large Standard Library.

Advantages

Presence of third-party modules. Extensive support libraries (NumPy for numerical calculations, Pandas for data analytics etc).Open source and community development. Easy to learn. User-friendly data structures. High-level language.

FUNCTIONS OF PYTHON

Functions are the most important aspect of an application. A function can be defined as the organized block of reusable code which can be called whenever required.

Python allows us to divide a large program into basic building blocks known as functions. The function contains a set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the Python program.

Functions help programmers break the program into smaller parts. They organize the code very effectively and avoid repetition. As the program grows, functions make the program more organized.

Python provides us with various inbuilt functions like range() or print(). Although, the user can create their own functions, which are called user-defined functions.

There are mainly two types of functions:

- **User-defined functions** – The user-defined functions are those defined by the user to perform a specific task.
- **Built-in functions** – The built-in functions are those functions that are pre-defined in Python.

LOGO AND FOUNDER O FPYTHON

MACHINE LEARNING

In iOS applications, we use Core ML to incorporate machine learning in iOS applications. It is Apple's framework to use pre-trained models in iOS applications. In this tutorial, we will discuss what machine learning is, different types of it, including some real-life examples of machine learning.

Machine learning is the field of study that allows computers to learn without being explicitly programmed. Using machine learning, we don't need to provide explicit instructions to computers for reacting to some special situations. We need to provide training to the computers to find real-time solutions for specific problems. The chess game is a famous example where machine learning is being used to play chess. The code lets the machine learn and optimizes itself over repeated games.

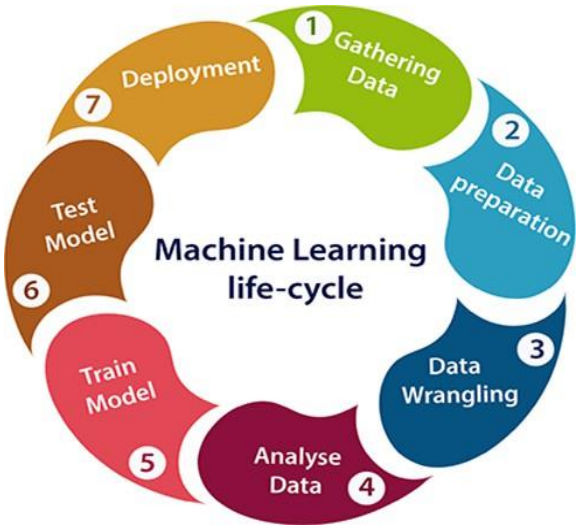
Machine Learning is broadly classified into two main categories.

- **Supervised Machine Learning**

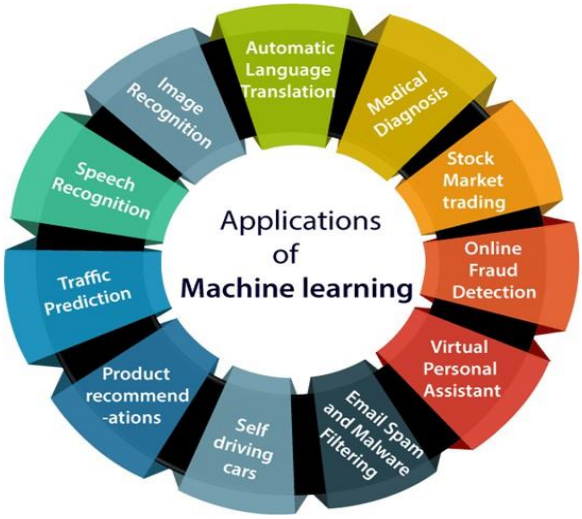
  Supervised learning is the type of machine learning in which machines are trained using well "labeled" training data, and on the basis of that data, machines predict the output. The labeled data means some input data is already tagged with the correct output.

- **Unsupervised Machine Learning**

  Unsupervised learning cannot be directly applied to a regression or classification problem because, unlike supervised learning, we have the input data but no corresponding output data. The goal of unsupervised learning is to find the underlying structure of the dataset, group that data according to similarities, and represent that dataset in a compressed format.



APPLICATION OF ML



LIFE CYCLE OF ML

MYSQL

MySQL is a relational database management system. It is open-source and free. Our MySQL tutorial includes all topics of MySQL database such as insert record, update record, delete record, select record, create table, drop table etc.

Features

MySQL is offered under two different editions: the open source MySQL Community Server and the proprietary Enterprise Server. MySQL Enterprise Server is differentiated by a series of

proprietary extensions which install as server plug-in, but otherwise shares the version numbering system and is built from the same code base.

- Major features as available in MySQL

• A broad subset of ANSI SQL 99, as well as extensions

• Cross-platform support

• Stored procedures, using a procedural language that closely adheres

to SQL/PSM[76]

• Triggers

• Cursors

• Updatable views

• Online Data Definition Language (DDL) when using the in no DB storage Engine.

• Information schema

Data types

• Integers.

• Booleans.

• Characters.

• Floating-point numbers.

Function

Stored function is a set of SQL statements that perform some operation and return a single value. Just like MySQL in-built function, it can be called from within a MySQL statement. By default, the stored function is associated with the default database.

4. MODULE DESCRIPTION

A modular design reduces system complexity, facilitates changes, and enables parallel development of different parts of the system. By dividing software into separately named and addressable components called modules, it becomes easier to implement and maintain. Each module can be developed and tested independently, with simplified interfaces for integration. Modularity allows a program to be intellectually manageable and supports software maintainability.

The effectiveness of a modular design can be evaluated using five key criteria: modular decomposability, modular composability, modular understandability, modular continuity, and modular protection.

Data Pre-processing

Data preprocessing is an essential step in building a machine learning model. It involves cleaning, transforming, and preparing the dataset to improve model accuracy and reliability. Validation techniques are used to estimate the error rate of the machine learning model, ensuring it is as close to the true error rate as possible.

In real-world scenarios, datasets may not perfectly represent the population, making preprocessing critical. Missing values, duplicate records, and variable types (integer, float, categorical) must be checked and handled carefully. Proper preprocessing ensures unbiased evaluation when tuning model hyperparameters.

Python libraries like Pandas provide tools for efficient data cleaning. Understanding the source of missing data helps decide how to fill or impute values. Common reasons for missing data include:

- Users forgetting to fill a field
- Data loss during manual transfer from legacy databases
- Programming errors
- Users intentionally leaving fields blank

Variable Identification

Variable identification can be performed through uni-variate, bi-variate, and multi-variate analysis. Steps include:

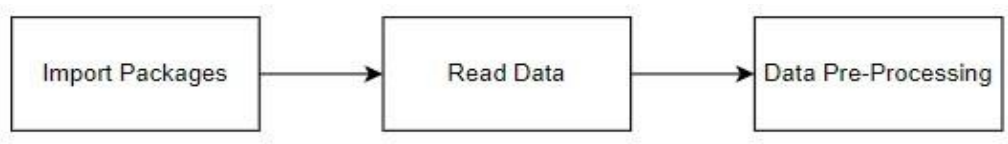
- Importing necessary libraries and loading the dataset
- Displaying the dataset in a DataFrame
- Checking dataset shape and column names
- Reviewing general properties and data types
- Identifying missing values and duplicate records
- Examining unique and count values
- Renaming and dropping unnecessary columns
- Creating extra derived columns as needed

Data Validation, Cleaning, and Preparation

After initial exploration, data cleaning and preparation ensure that the dataset is ready for modeling. Validation sets, held back from training, are used to estimate model performance while tuning hyperparameters. Cleaning may include renaming columns, removing duplicates, handling missing values, and converting data types.

The primary goal of data cleaning is to detect and remove errors and anomalies, increasing the value of data for analysis and decision-making.

Module Diagram



Exploratory Data Analysis and Visualization

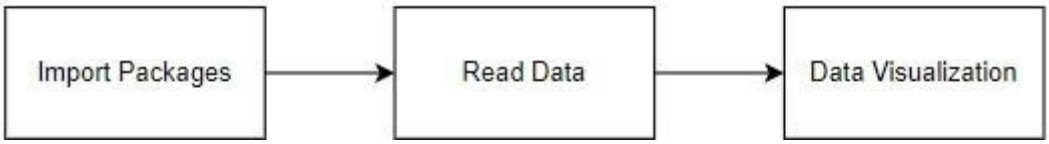
Data visualization provides a qualitative understanding of the dataset and is an essential skill in applied statistics and machine learning. It helps identify patterns, outliers, and corrupt data while making complex information easier to interpret for stakeholders. Visualizations also reveal relationships between variables that may not be obvious from raw data.

Common visualization techniques include:

- Line plots for time series data
- Bar charts for categorical quantities
- Histograms and box plots to summarize distributions

Visualizations not only aid exploration but also support better modeling decisions. Proper use of Python visualization libraries allows efficient exploration and understanding of datasets.

Module Diagram



1. Dataset Report

The dataset used for this project is **Loan_Train.csv**, which contains historical loan applicant information. It includes demographic details, income details, credit history, and loan status.

- Total Records: 600+ entries
- Features: 11 input attributes
- Target Variable: Loan Status
- Type: Supervised Classification Dataset

The dataset was analysed for missing values, categorical variables, and outliers before training the model.

2. Data Pre-processing Report

The following pre-processing steps were performed:

- Missing values were handled using forward fill method.
- Categorical variables were encoded using Label Encoding.
- Dummy variable trap was avoided.

- Data was split into training (80%) and testing (20%).
- Outliers were removed using IQR method (if applied).

These steps ensured improved model performance and accuracy.

3. Model Training Report

Multiple machine learning algorithms were tested:

- Logistic Regression
- K-Nearest Neighbours
- Support Vector Machine
- Naive Bayes
- Decision Tree
- Random Forest
- Gradient Boosting

Among all, **Random Forest Classifier** provided the highest accuracy and stable performance.

4. Model Performance Report

Evaluation was done using:

- Accuracy Score
- Confusion Matrix
- Training vs Testing comparison

The final model achieved good accuracy on unseen test data, indicating proper generalization and reduced overfitting.

5. Deployment Report

The trained model was saved using Pickle (`loan_model.pkl`).

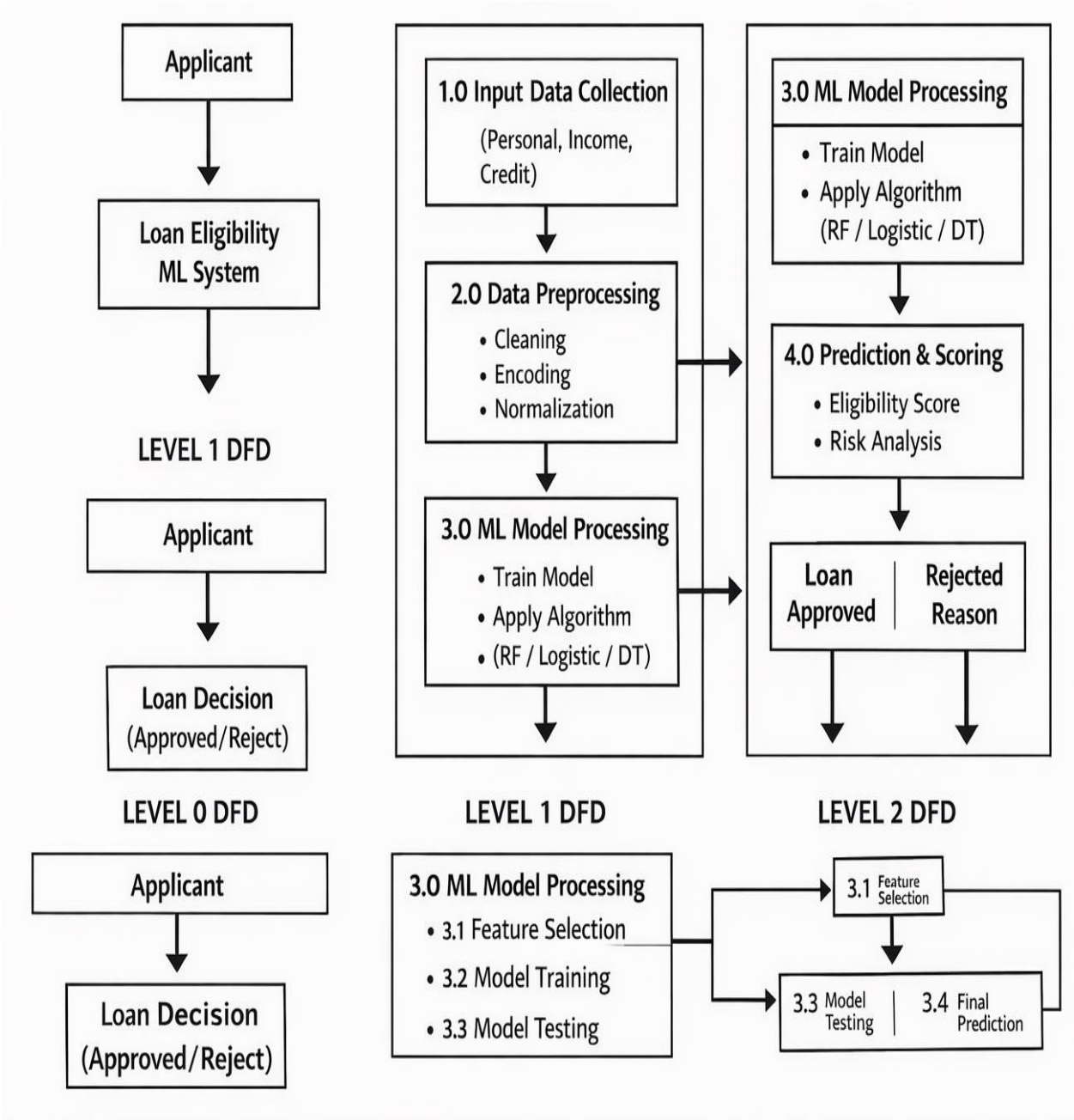
A Flask web application was created to:

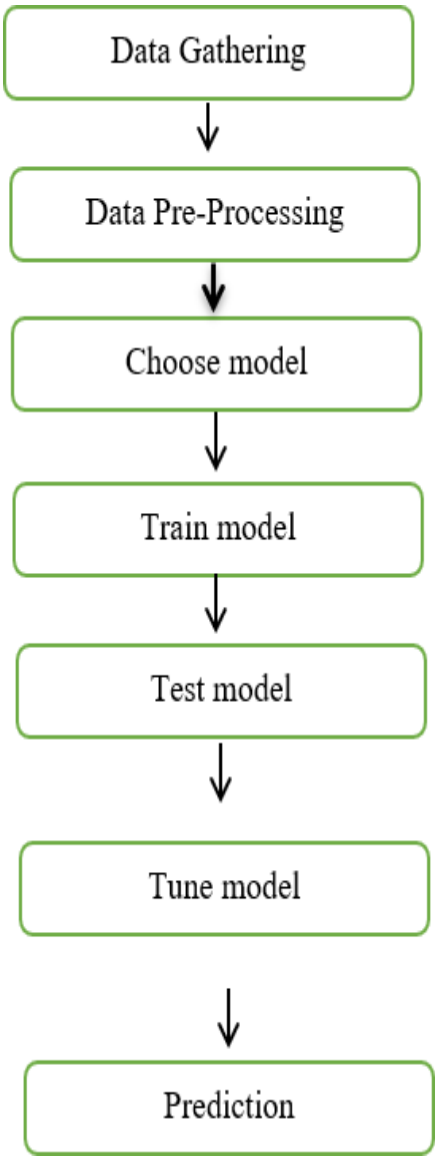
- Accept user inputs
- Process data
- Load trained model
- Predict loan approval status
- Display result on web page

This makes the system interactive and user-friendly.

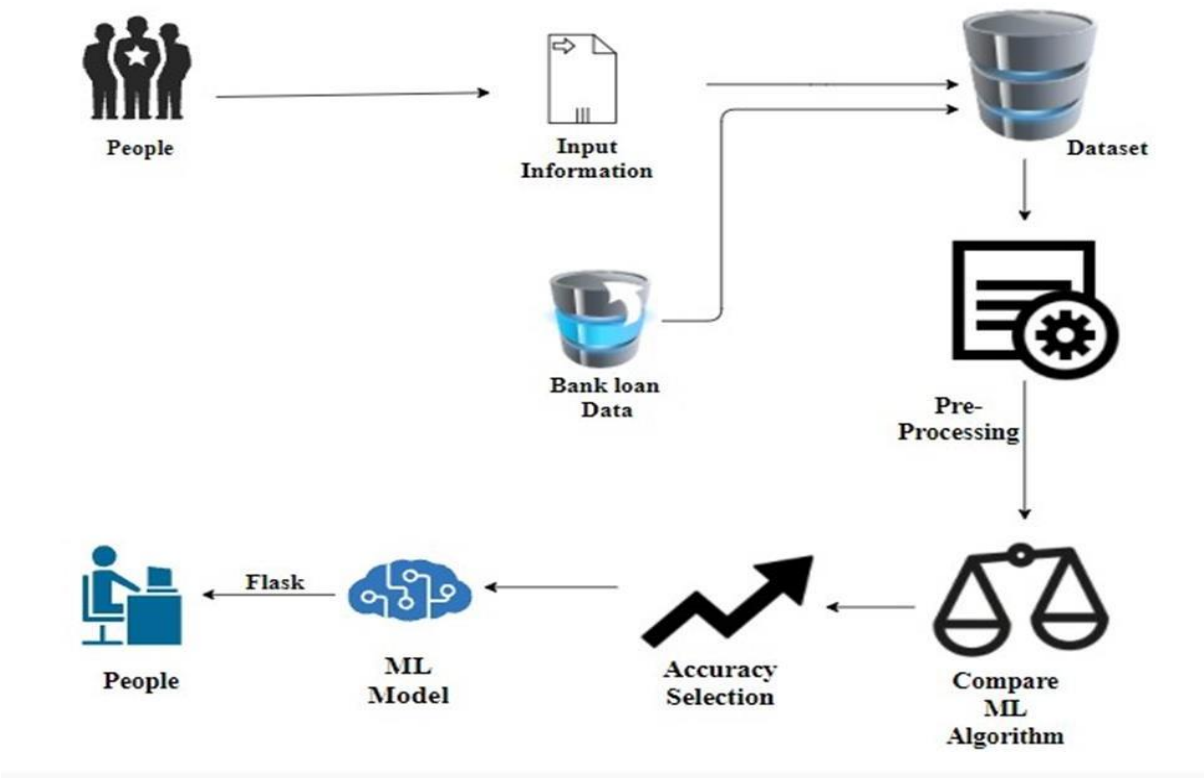
5. SYSTEM DEIGN:

5.1 DATA FLOW DIAGRAM:

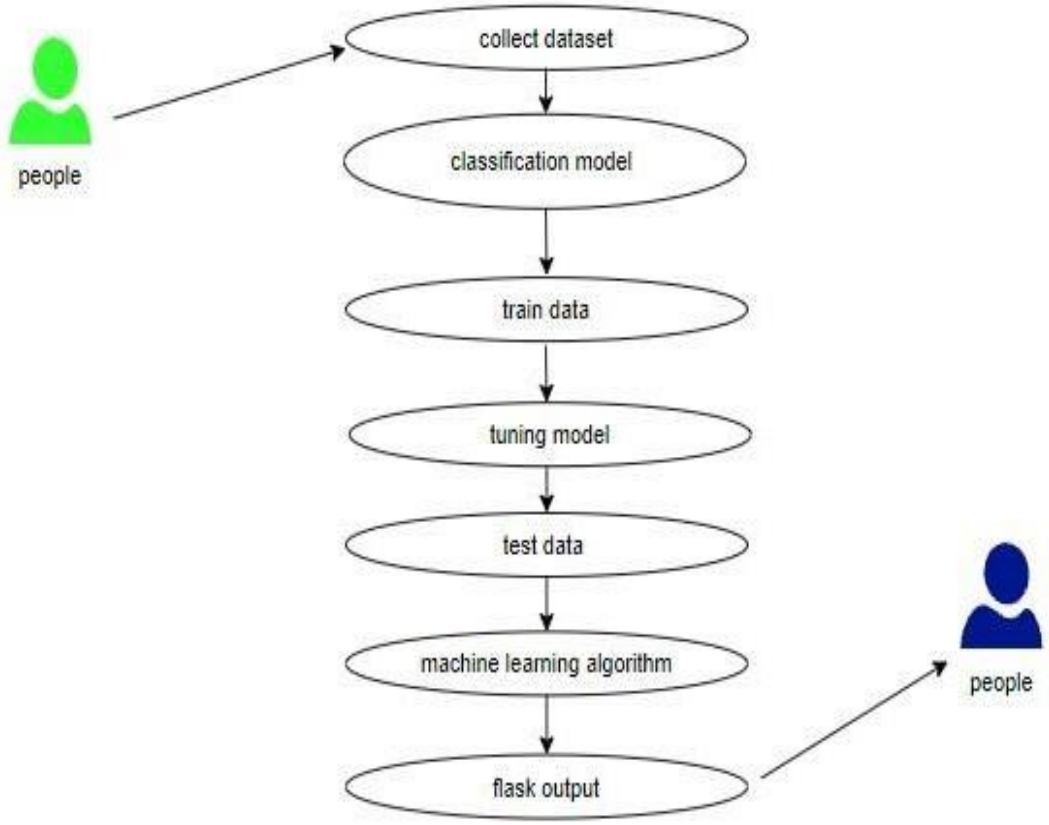



Data Gathering
Data Pre-Processing
Choose model
Train model
Test model
Tune model
Prediction

5.2 ARCHITECTURE DIAGRAM:



5.3 USECASE DIAGRAM:



6. SYSTEM IMPLEMENTATION

6.1 SCREEN LAYOUT DESIGN:

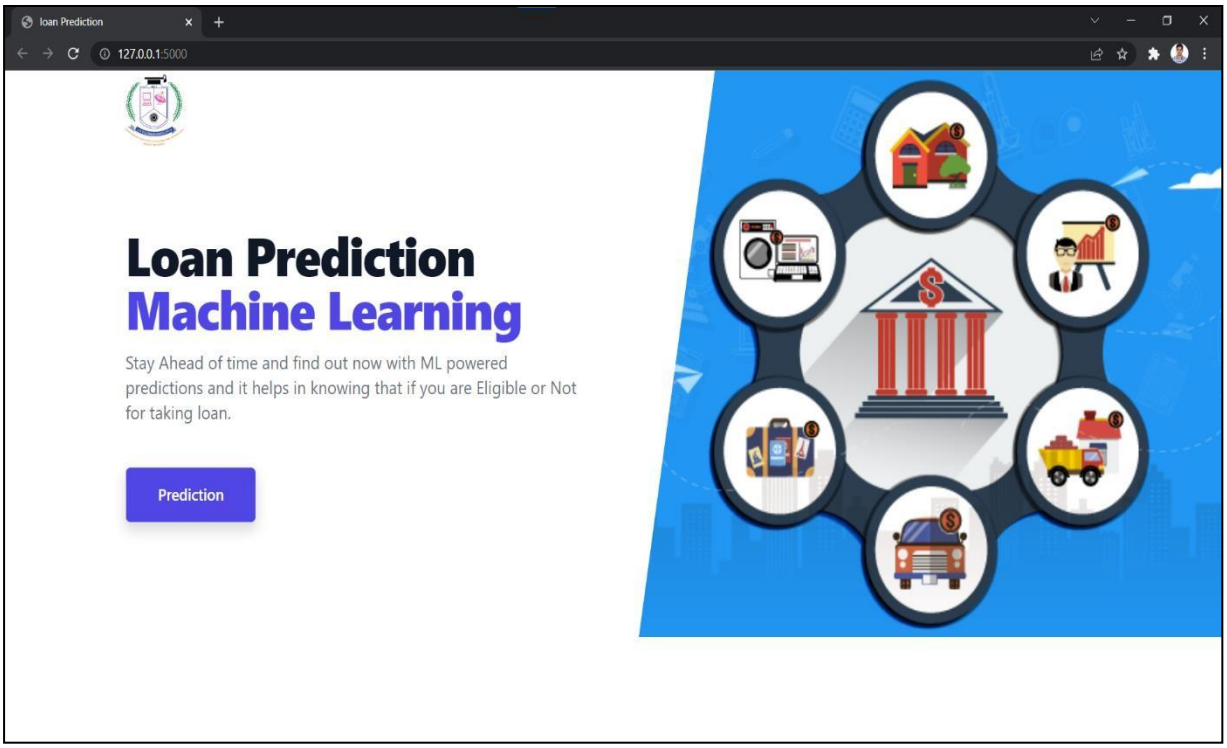


Fig:6.1:HOMEPAGE

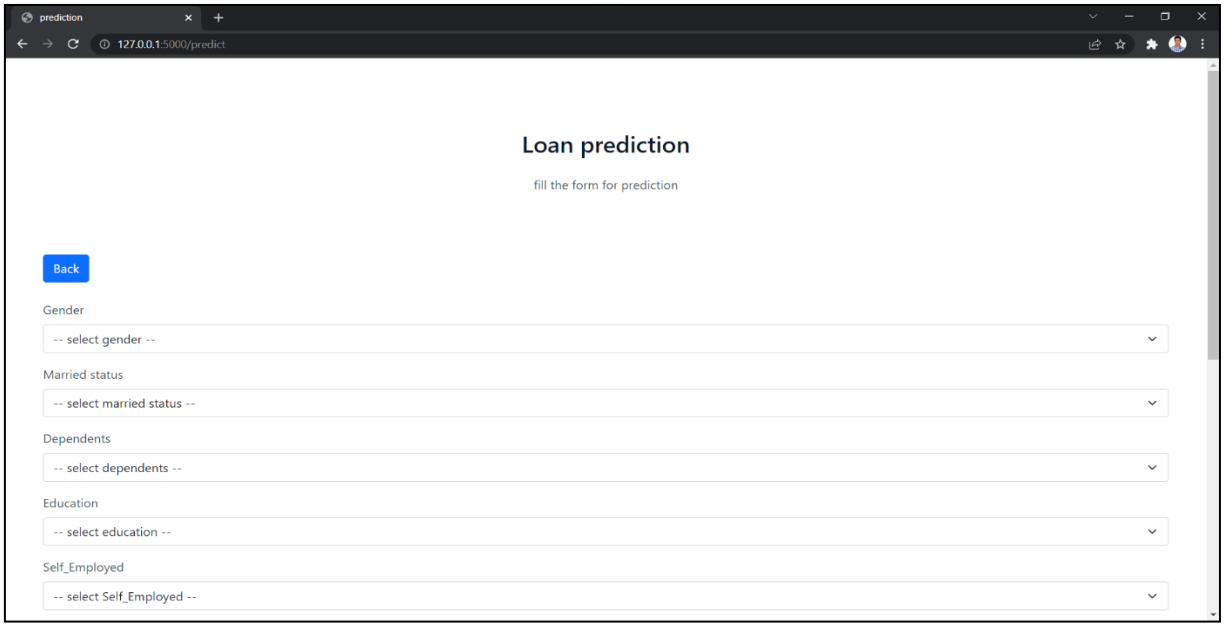


Fig:6.2:INPUTS PAGE

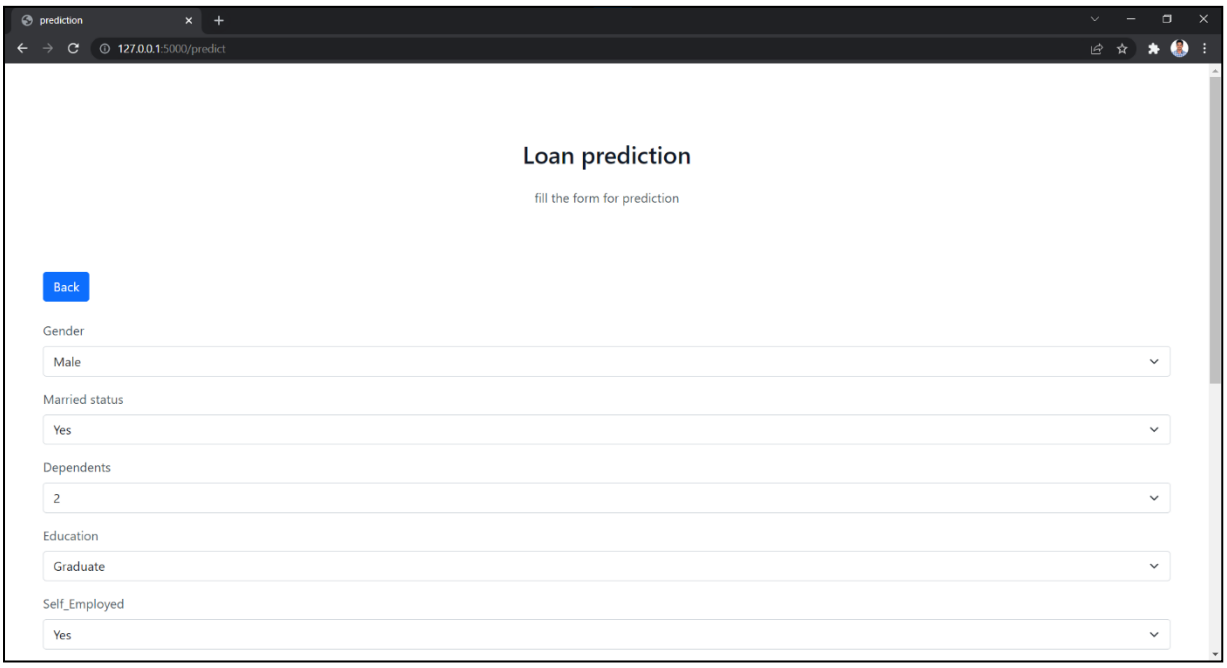


Fig:6.3:INPUTS GIVEN BY THE USER

Fig:6.4: LOAN APPROVED

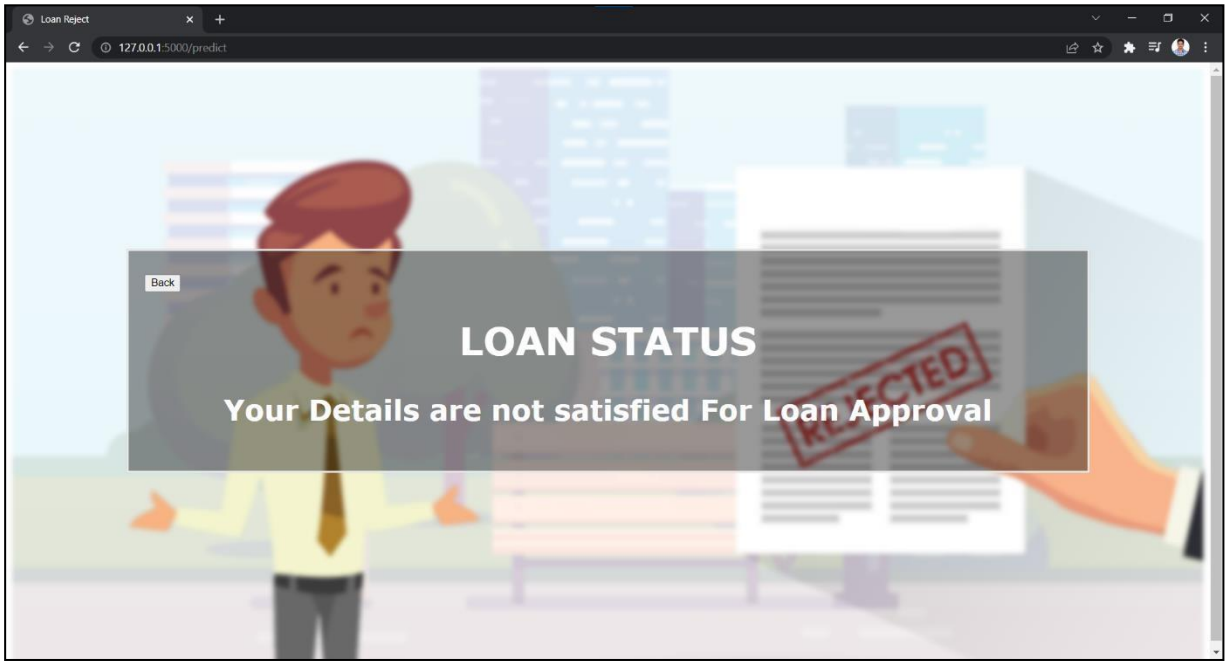
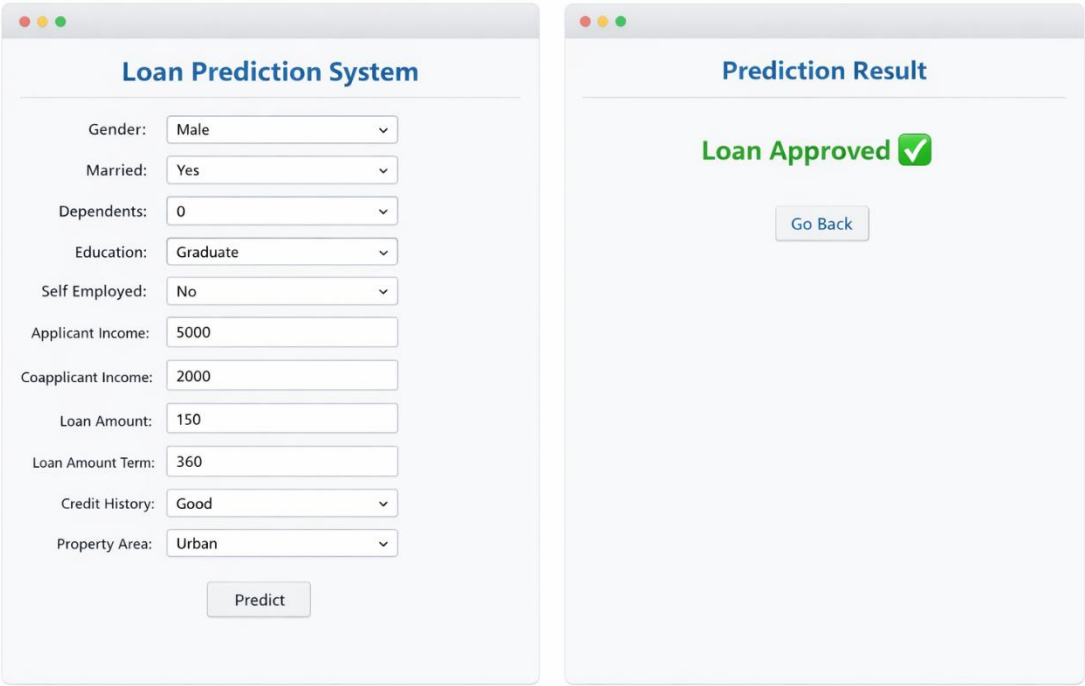
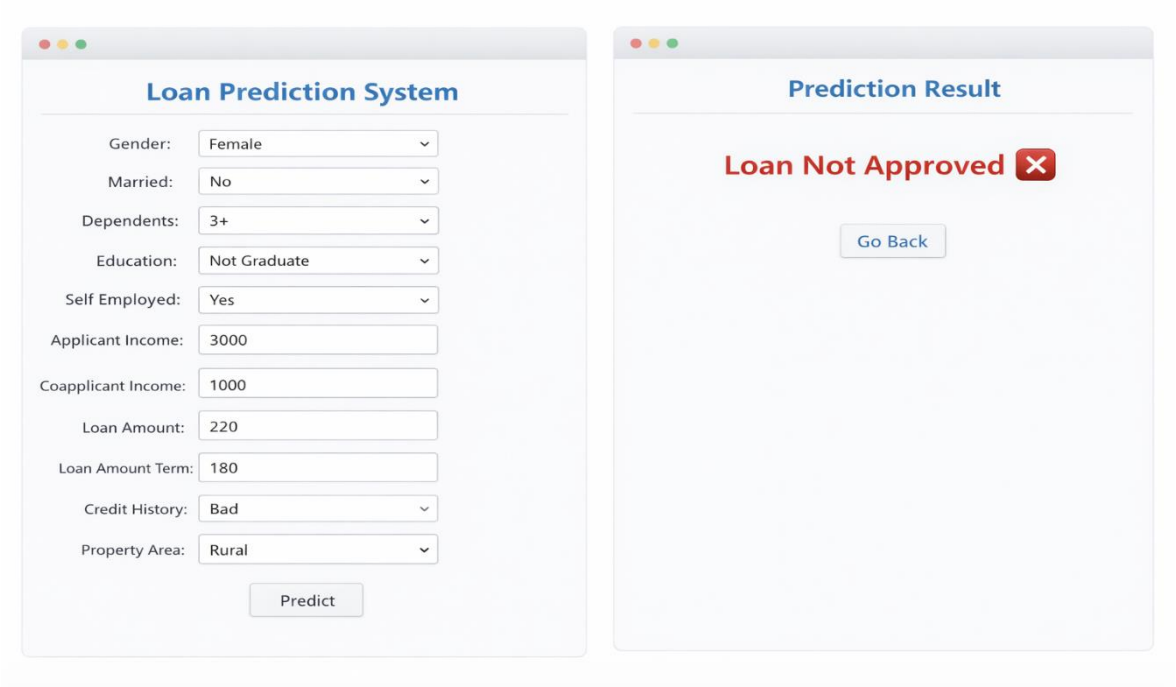
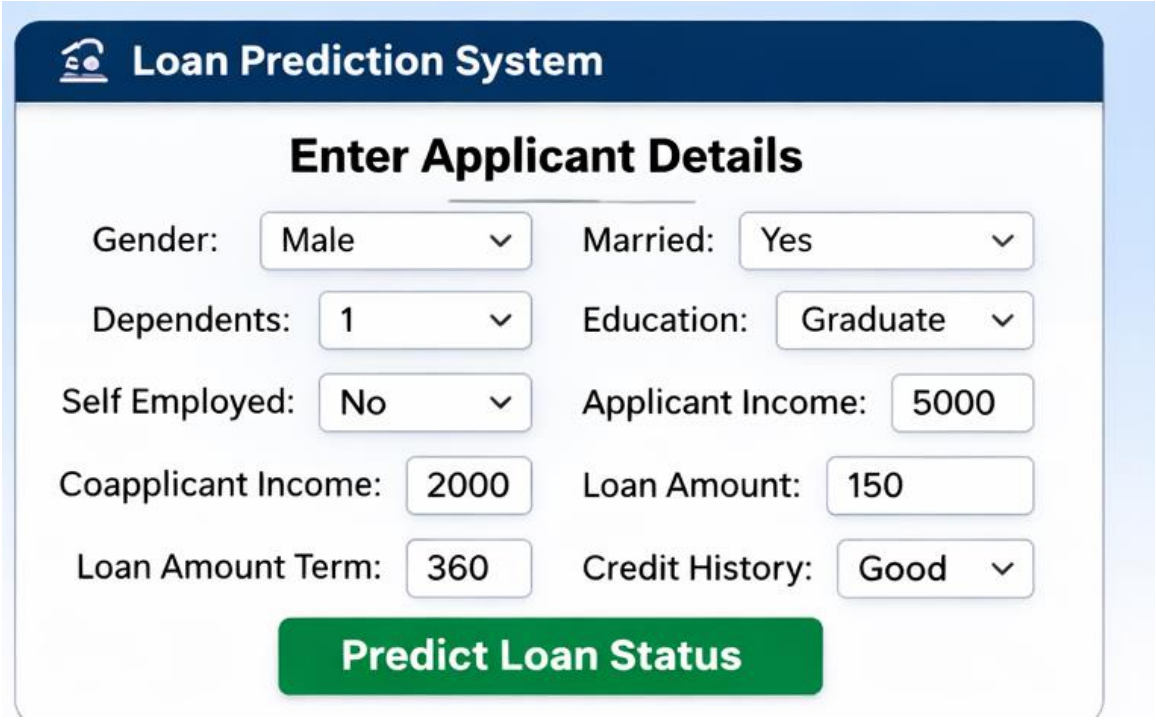
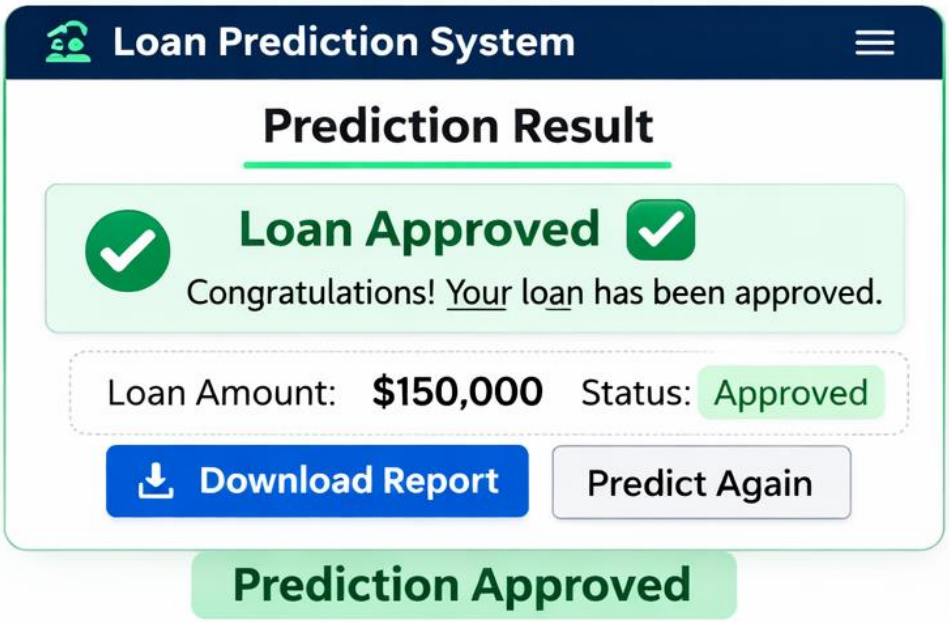


Fig:6.5:LOAN REJECTED




Loan Prediction System
Gender: Female
Married: No
Dependents: 3+
Education: Not Graduate
Self Employed: Yes
Applicant Income: 3000
Coapplicant Income: 1000
Loan Amount: 220
Loan Amount Term: 180
Credit History: Bad
Property Area: Rural
Predict
Prediction Result
Loan Not Approved
Go Back


Loan Prediction System
Enter Applicant Details
Gender: Male
Married: Yes
Dependents: 1
Education: Graduate
Self Employed: No
Applicant Income: 5000
Coapplicant Income: 2000
Loan Amount: 150
Loan Amount Term: 360
Credit History: Good
Predict Loan Status


Loan Prediction System
Prediction Result
Loan Approved
Congratulations! Your loan has been approved.
Loan Amount: $150,000 Status: Approved
Download Report
Predict Again
Prediction Approved

6.2 SAMPLE SOURCE CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import pickle
import warnings

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import precision_score, recall_score, f1_score

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC

warnings.filterwarnings("ignore")

# ---------------------------
# 2. Load Dataset
# ---------------------------

print("Loading Dataset...")
data = pd.read_csv("loan_data.csv") # Replace with your dataset file

print("Dataset Shape:", data.shape)
print("\nFirst 5 Records:\n", data.head())

# ---------------------------
# 3. Data Preprocessing
# ---------------------------
```

```
print("\nChecking Missing Values:\n", data.isnull().sum())

# Fill missing numerical values with median
data['LoanAmount'].fillna(data['LoanAmount'].median(), inplace=True)
data['Loan_Amount_Term'].fillna(data['Loan_Amount_Term'].median(), inplace=True)
data['ApplicantIncome'].fillna(data['ApplicantIncome'].median(), inplace=True)
data['CoapplicantIncome'].fillna(data['CoapplicantIncome'].median(), inplace=True)

# Fill categorical missing values with mode
data['Gender'].fillna(data['Gender'].mode()[0], inplace=True)
data['Married'].fillna(data['Married'].mode()[0], inplace=True)
data['Dependents'].fillna(data['Dependents'].mode()[0], inplace=True)
data['Self_Employed'].fillna(data['Self_Employed'].mode()[0], inplace=True)
data['Credit_History'].fillna(data['Credit_History'].mode()[0], inplace=True)
data['Property_Area'].fillna(data['Property_Area'].mode()[0], inplace=True)

print("\nMissing Values After Handling:\n", data.isnull().sum())

# ---------------------------
# 4. Encoding Categorical Data
# ---------------------------

label_encoder = LabelEncoder()

categorical_cols = ['Gender', 'Married', 'Dependents',
'Education', 'Self_Employed',
'Property_Area', 'Loan_Status']

for col in categorical_cols:
data[col] = label_encoder.fit_transform(data[col])

# ---------------------------
# 5. Feature Selection
# ---------------------------
```

```
X = data.drop(['Loan_ID', 'Loan_Status'], axis=1)
y = data['Loan_Status']

# ---------------------------
# 6. Feature Scaling
# ---------------------------

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ---------------------------
# 7. Train-Test Split
# ---------------------------

X_train, X_test, y_train, y_test = train_test_split(
X_scaled, y, test_size=0.25, random_state=42)

print("\nTraining Size:", X_train.shape)
print("Testing Size:", X_test.shape)

plt.figure()
sns.countplot(x='Loan_Status', data=data)
plt.title("Loan Approval Distribution")
plt.show()

plt.figure()
sns.heatmap(data.corr(), annot=False)
plt.title("Correlation Heatmap")
plt.show()

models = {
"Logistic Regression": LogisticRegression(),
"Decision Tree": DecisionTreeClassifier(),
```

```
"Random Forest": RandomForestClassifier(n_estimators=200),
"Gradient Boosting": GradientBoostingClassifier(),
"Support Vector Machine": SVC()
}

results = {}

print("\nTraining Models...\n")

for name, model in models.items():
model.fit(X_train, y_train)
predictions = model.predict(X_test)

acc = accuracy_score(y_test, predictions)
prec = precision_score(y_test, predictions)
rec = recall_score(y_test, predictions)
f1 = f1_score(y_test, predictions)

results[name] = acc

print("Model:", name)
print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1 Score:", f1)
print("Confusion Matrix:\n", confusion_matrix(y_test, predictions))
print("-" * 50)

best_model_name = max(results, key=results.get)
print("\nBest Performing Model:", best_model_name)

best_model = models[best_model_name]

cv_scores = cross_val_score(best_model, X_scaled, y, cv=5)
```

```
print("\nCross Validation Accuracy:", cv_scores.mean())

pickle.dump(best_model, open("loan_model.pkl", "wb"))
pickle.dump(scaler, open("scaler.pkl", "wb"))

print("\nModel and Scaler Saved Successfully!")


# Example New Applicant Data
# (Make sure order matches dataset features)

sample_input = np.array([[1, 1, 0, 1, 0, 5000, 2000,
150, 360, 1, 2]])

sample_input_scaled = scaler.transform(sample_input)

prediction = best_model.predict(sample_input_scaled)

if prediction[0] == 1:
print("\nLoan Status: APPROVED")
else:
print("\nLoan Status: REJECTED")

import numpy as np
importpandasaspd

importmatplotlib.pyplotasplt import
missingno as mso

importseabornassns

fromsklearn.model_selectionimporttrain_test_split from
imblearn.over_sampling import SMOTE

fromsklearn.linear_modelimportLogisticRegression from
sklearn.neighbors import KNeighborsClassifier from
sklearn.svm import SVC
```

```
fromsklearn.naive_bayesimportGaussianNB

from sklearn.tree import DecisionTreeClassifier
fromsklearn.ensembleimportRandomForestClassifier

fromsklearn.ensembleimportGradientBoostingClassifier

df=pd.read_csv("Loan_Train.csv")

df.head()

countMale = len(df[df.Gender == 'Male'])
countFemale=len(df[df.Gender=='Female']) countNull =
len(df[df.Gender.isnull()])


print("PercentageofMaleapplicant:{:.2f}%".format((countMale/ (len(df.Gender))*100)))

print("PercentageofFemaleapplicant:{:.2f}%".format((countFemale/ (len(df.Gender))*100)))

print("Missingvaluespercentage:{:.2f}%".format((countNull/

(len(df.Gender))*100)))

df.Married.value_counts(dropna=False)

sns.countplot(x="Married",data=df,palette="Paired") plt.show()

countMale = len(df[df.Gender == 'Male'])
countFemale = len(df[df.Gender == 'Female'])
countNull = len(df[df.Gender.isnull()])

df=pd.get_dummies(df)

df=df.drop(['Gender_Female','Married_No','Education_NotGraduate', 'Self_Employed_No',
                    'Loan_Status_N'], axis = 1)

Q1=df.quantile(0.25)
Q3=df.quantile(0.75)

IQR=Q3-Q1
```

```
df = df[~((df < (Q1 - 1.5 * IQR)) |(df > (Q3 + 1.5 * IQR))).any(axis=1)]
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,

random_state=0)

LRclassifier=LogisticRegression(solver='saga',max_iter=500,random_state=1)
LRclassifier.fit(X_train, y_train)

y_pred=LRclassifier.predict(X_test)

scoreListknn=[]

foriinrange(1,21):

    KNclassifier=KNeighborsClassifier(n_neighbors=i) KNclassifier.fit(X_train, y_train)

    scoreListknn.append(KNclassifier.score(X_test,y_test))

SVCclassifier=SVC(kernel='rbf',max_iter=500)
SVCclassifier.fit(X_train, y_train)

y_pred=SVCclassifier.predict(X_test)

NBclassifier2 = GaussianNB()
NBclassifier2.fit(X_train,y_train)

y_pred=NBclassifier2.predict(X_test)

scoreListDT= []

foriinrange(2,21):

    DTclassifier=DecisionTreeClassifier(max_leaf_nodes=i)

    DTclassifier.fit(X_train, y_train) scoreListDT.append(DTclassifier.score(X_test,y_test))

scoreListRF= []

foriinrange(2,25):

    RFclassifier=RandomForestClassifier(n_estimators=1000,random_state= 1,
max_leaf_nodes=i)

    RFclassifier.fit(X_train,y_train)
```

```
    scoreListRF.append(RFclassifier.score(X_test,y_test))

GBclassifier=GradientBoostingClassifier(subsample=1,n_estimators=200, max_depth=5,
max_leaf_nodes=40)

GBclassifier.fit(X_train,y_train)

y_pred=GBclassifier.predict(X_test)

compare = pd.DataFrame({'Model': ['Logistic Regression', 'Random Forest','K
Neighbors','DecisionTree','SVM','GaussianNB','GradientBoost'],'Accuracy':
[LRAcc*100,RFAcc*100, KNAcc*100, DTAcc*100, SVCAcc*100,
NBAcc2*100,GBAcc*100]})

compare.sort_values(by='Accuracy',ascending=False)

import pickle

import numpy as np
import pandas as pd
import pickle

from sklearn.model_selection import
train_test_split
from sklearn.ensemble import
RandomForestClassifier
from sklearn.preprocessing import
LabelEncoder
from sklearn.metrics import
accuracy_score

le = LabelEncoder()

categorical_columns = [
'Gender', 'Married', 'Education',
'Self_Employed', 'Property_Area',
'Loan_Status'
]
```

```
SVCclassifier=SVC(kernel='rbf',max_iter=500)
SVCclassifier.fit(X_train, y_train)

y_pred=SVCclassifier.predict(X_test)

NBclassifier2 = GaussianNB()
NBclassifier2.fit(X_train,y_train)

y_pred=NBclassifier2.predict(X_test)

scoreListDT= []

foriinrange(2,21):

    DTclassifier=DecisionTreeClassifier(max_leaf_nodes=i)

    DTclassifier.fit(X_train, y_train) scoreListDT.append(DTclassifier.score(X_test,y_test))

scoreListRF= []

foriinrange(2,25):

    RFclassifier=RandomForestClassifier(n_estimators=1000,random_state= 1,
max_leaf_nodes=i)

    RFclassifier.fit(X_train,y_train)

    scoreListRF.append(RFclassifier.score(X_test,y_test))

GBclassifier=GradientBoostingClassifier(subsample=1,n_estimators=200, max_depth=5,
max_leaf_nodes=40)

GBclassifier.fit(X_train,y_train)

y_pred=GBclassifier.predict(X_test)

compare = pd.DataFrame({'Model': ['Logistic Regression', 'Random Forest','K
Neighbors','DecisionTree','SVM','GaussianNB','GradientBoost'],'Accuracy':
[LRAcc*100,RFAcc*100, KNAcc*100, DTAcc*100, SVCAcc*100,
NBAcc2*100,GBAcc*100]})

compare.sort_values(by='Accuracy',ascending=False)
```

```
for col in categorical_columns:
df[col] = le.fit_transform(df[col])

X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]

X_train, X_test, y_train, y_test =
train_test_split(
X, y, test_size=0.2, random_state=42

model =
RandomForestClassifier(n_estimators=20
0, random_state=42)
model.fit(X_train, y_train)

file=open("RFclassifier.pkl",'wb')

from flask import Flask, render_template,
request
import numpy as np
import pickle


# Initialize app
app = Flask(__name__)


# Load trained model
model =
pickle.load(open("loan_model.pkl", "rb"))

@app.route("/predict", methods=["POST"])
def predict():


Gender = int(request.form["Gender"])
Married = int(request.form["Married"])
Dependents = int(request.form["Dependents"])
Education = int(request.form["Education"])
```

```
Self_Employed = int(request.form["Self_Employed"])
ApplicantIncome = float(request.form["ApplicantIncome"])
CoapplicantIncome =
float(request.form["CoapplicantIncome"])
LoanAmount = float(request.form["LoanAmount"])
Loan_Amount_Term =
float(request.form["Loan_Amount_Term"])
Credit_History = float(request.form["Credit_History"])
Property_Area = int(request.form["Property_Area"])

input_data = np.array([[Gender, Married, Dependents,
Education, Self_Employed,
ApplicantIncome, CoapplicantIncome,
LoanAmount, Loan_Amount_Term,
Credit_History, Property_Area]])

prediction = model.predict(input_data)

if prediction[0] == 1:
result = "Loan Approved ✅"
else:
result = "Loan Not Approved ❌"
SVCclassifier=SVC(kernel='rbf',max_iter=500)
SVCclassifier.fit(X_train, y_train)

y_pred=SVCclassifier.predict(X_test)

NBclassifier2 = GaussianNB()
NBclassifier2.fit(X_train,y_train)

y_pred=NBclassifier2.predict(X_test)

scoreListDT= []

foriinrange(2,21):

    DTclassifier=DecisionTreeClassifier(max_leaf_nodes=i)
```

```
    DTclassifier.fit(X_train, y_train) scoreListDT.append(DTclassifier.score(X_test,y_test))

scoreListRF= []

foriinrange(2,25):

    RFclassifier=RandomForestClassifier(n_estimators=1000,random_state= 1,
max_leaf_nodes=i)

    RFclassifier.fit(X_train,y_train)

    scoreListRF.append(RFclassifier.score(X_test,y_test))

GBclassifier=GradientBoostingClassifier(subsample=1,n_estimators=200, max_depth=5,
max_leaf_nodes=40)

GBclassifier.fit(X_train,y_train)

y_pred=GBclassifier.predict(X_test)

compare = pd.DataFrame({'Model': ['Logistic Regression', 'Random Forest','K
Neighbors','DecisionTree','SVM','GaussianNB','GradientBoost'],'Accuracy':
[LRAcc*100,RFAcc*100, KNAcc*100, DTAcc*100, SVCAcc*100,
NBAcc2*100,GBAcc*100]})

compare.sort_values(by='Accuracy',ascending=False)


return render_template("result.html",
prediction_text=result)

pickle.dump(RFclassifier,file)

<!doctypehtml>

<htmllang="en">

  <head><title>loanPrediction</title></head><body>

        <spanclass="blockxl:inline">LoanPrediction</span>

        <spanclass="blocktext-indigo-600xl:inline">MachineLearning</span>
```

```
</h1><pclass="mt-3text-basetext-gray-500sm:mt-5sm:text-lgsm:max- w-xl
sm:mx-auto md:mt-5 md:text-xl lg:mx-0">Stay Ahead of time and find out now with ML
powered predictions and it helps in knowing that if you are Eligible or Not for taking
loan.</p><div class="mt-5 sm:mt-8 sm:flex sm:justify-center lg:justify-start"><div
class="rounded-md shadow"><a href="./predict"class="w-full flex items-center justify-
center px-8 py-3 border border-transparent text-base font-medium rounded-md text-white
bg- indigo-600 hover:bg-indigo-700 md:py-4 md:text-lg md:px-
10">Prediction</a></div></div></div>

</body></html>

<!doctypehtml>

<htmllang="en">

  <head><title>prediction</title></head><body>

  <sectionclass="text-gray-600body-font"><divclass="containerpx-5py-24 mx-
auto"><div class="flex flex-col text-center w-full mb-20">

<h1 class="sm:text-3xl text-2xl font-medium title-font mb-4 text-gray-
900">Loanprediction</h1><pclass="lg:w-2/3mx-autoleading-relaxedtext- base">fill the
form for prediction</p></div><div></div>

    <aclass="btnbtn-primary"href="./"role="button">Back</a>

  </br></br><formaction='/predict'method='POST'><divclass="mb-3">

  <labelfor="exampleFormControlInput1"class="form-label">Gender</label>

  <selectclass="form-select"id="gender"name="gender"aria-label="Default select
example"><option selected>-- select gender --</option>

  <optionvalue="Male">Male</option><optionvalue="Female">Female</option>

</select></div><divclass="mb-3">

<label for="exampleFormControlInput1"class="form-label">Enter
ApplicantIncome</label><input type="text"class="form-
control"id="ApplicantIncome"name="ApplicantIncome"placeholder="ApplicantIncome">

</div></div><buttontype="submit"class="btnbtn-primary">Predict</button>
```

```
</form></div></section>

fromflaskimportFlask,escape,request,render_template import pickle

model=pickle.load(open('RFclassifier.pkl','rb')
) @app.route('/')

defhome():

    return
render_template("index.html")
@app.route('/predict',methods=['
GET','POST']) def predict():

        arr=np.array([[credit,ApplicantIncome,CoapplicantIncome,Lo
anAmount, Loan_Amount_Term, Gender_Male, Married_Yes,
dependents_0, dependents_1, dependents_2, dependents_3,
education_graduate, self_employed_yes,semiurban, urban, rural ]])

        prediction=model.predi
        ct(arr)
        if(prediction==0):

            returnrender_template("LR.htm
        l") elif(prediction!=0):

            returnrender_template("LA.html")

    else:

        returnrender_template("
prediction.html") if name==
"main":

    app.run(debug=True)

import pandas as pd
import numpy as np
import pickle

from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv("Loan_Train.csv")

# Fill missing values
df.fillna(method='ffill', inplace=True)

# Encode categorical columns
le = LabelEncoder()

categorical_columns = [
'Gender',
'Married',
'Dependents',
'Education',
'Self_Employed',
'Property_Area',
'Loan_Status'
]

for col in categorical_columns:
df[col] = le.fit_transform(df[col])

# Split data
X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
```

```
# Train Random Forest
model = RandomForestClassifier(n_estimators=300, random_state=42)
model.fit(X_train, y_train)

# Accuracy
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy * 100)

# Save model
with open("loan_model.pkl", "wb") as file:
pickle.dump(model, file)

print("Model Saved Successfully!")

from flask import Flask, render_template, request
import numpy as np
import pickle

app = Flask(__name__)

# Load trained model
model = pickle.load(open("loan_model.pkl", "rb"))

@app.route("/")
def home():
return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():

Gender = int(request.form["Gender"])
Married = int(request.form["Married"])
```

```
Dependents = int(request.form["Dependents"])
Education = int(request.form["Education"])
Self_Employed = int(request.form["Self_Employed"])
ApplicantIncome = float(request.form["ApplicantIncome"])
CoapplicantIncome = float(request.form["CoapplicantIncome"])
LoanAmount = float(request.form["LoanAmount"])
Loan_Amount_Term = float(request.form["Loan_Amount_Term"])
Credit_History = float(request.form["Credit_History"])
Property_Area = int(request.form["Property_Area"])

data = np.array([[Gender, Married, Dependents,
Education, Self_Employed,
ApplicantIncome, CoapplicantIncome,
LoanAmount, Loan_Amount_Term,
Credit_History, Property_Area]])

prediction = model.predict(data)

if prediction[0] == 1:
result = "Loan Approved ✅"
else:
result = "Loan Not Approved ❌"

return render_template("result.html", prediction_text=result)

if __name__ == "__main__":
app.run(debug=True)
```

```
<!DOCTYPE html>
<html>
<head>
<title>Loan Prediction</title>
</head>
<body>
```

```
<h2>Loan Prediction System</h2>

<form action="/predict" method="post">
SVCclassifier=SVC(kernel='rbf',max_iter=500)
SVCclassifier.fit(X_train, y_train)

y_pred=SVCclassifier.predict(X_test)

NBclassifier2 = GaussianNB()
NBclassifier2.fit(X_train,y_train)

y_pred=NBclassifier2.predict(X_test)

scoreListDT= []

foriinrange(2,21):

    DTclassifier=DecisionTreeClassifier(max_leaf_nodes=i)

    DTclassifier.fit(X_train, y_train) scoreListDT.append(DTclassifier.score(X_test,y_test))

scoreListRF= []

foriinrange(2,25):

    RFclassifier=RandomForestClassifier(n_estimators=1000,random_state= 1,
max_leaf_nodes=i)

    RFclassifier.fit(X_train,y_train)

    scoreListRF.append(RFclassifier.score(X_test,y_test))

GBclassifier=GradientBoostingClassifier(subsample=1,n_estimators=200, max_depth=5,
max_leaf_nodes=40)

GBclassifier.fit(X_train,y_train)

y_pred=GBclassifier.predict(X_test)

compare = pd.DataFrame({'Model': ['Logistic Regression', 'Random Forest','K
Neighbors','DecisionTree','SVM','GaussianNB','GradientBoost'],'Accuracy':
[LRAcc*100,RFAcc*100, KNAcc*100, DTAcc*100, SVCAcc*100,
NBAcc2*100,GBAcc*100]})
```

```
compare.sort_values(by='Accuracy',ascending=False)
```

Gender:

```
<select name="Gender">
<option value="1">Male</option>
<option value="0">Female</option>
</select><br><br>
```

Married:

```
<select name="Married">
<option value="1">Yes</option>
<option value="0">No</option>
</select><br><br>
```

Dependents:

```
<select name="Dependents">
<option value="0">0</option>
<option value="1">1</option>
<option value="2">2</option>
<option value="3">3+</option>
</select><br><br>
```

Education:

```
<select name="Education">
<option value="1">Graduate</option>
<option value="0">Not Graduate</option>
</select><br><br>
```

Self Employed:

```
<select name="Self_Employed">
<option value="1">Yes</option>
<option value="0">No</option>
</select><br><br>
```

Applicant Income:

```
<input type="text" name="ApplicantIncome"><br><br>

Coapplicant Income:
<input type="text" name="CoapplicantIncome"><br><br>

Loan Amount:
<input type="text" name="LoanAmount"><br><br>

Loan Amount Term:
<input type="text" name="Loan_Amount_Term"><br><br>

Credit History:
<select name="Credit_History">
<option value="1">Good</option>
<option value="0">Bad</option>
</select><br><br>

Property Area:
<select name="Property_Area">
<option value="0">Rural</option>
<option value="1">Semiurban</option>
<option value="2">Urban</option>
</select><br><br>

<input type="submit" value="Predict">

</form>

</body>
</html>
# Save model
with open("loan_model.pkl", "wb") as file:
pickle.dump(model, file)
```

```
print("Model Saved Successfully!")

from flask import Flask, render_template, request
import numpy as np
import pickle

app = Flask(__name__)

# Load trained model
model = pickle.load(open("loan_model.pkl", "rb"))
data['LoanAmount'].fillna(data['LoanAmount'].median(), inplace=True)
data['Loan_Amount_Term'].fillna(data['Loan_Amount_Term'].median(), inplace=True)
data['ApplicantIncome'].fillna(data['ApplicantIncome'].median(), inplace=True)
data['CoapplicantIncome'].fillna(data['CoapplicantIncome'].median(), inplace=True)

# Fill categorical missing values with mode
data['Gender'].fillna(data['Gender'].mode()[0], inplace=True)
data['Married'].fillna(data['Married'].mode()[0], inplace=True)
data['Dependents'].fillna(data['Dependents'].mode()[0], inplace=True)
data['Self_Employed'].fillna(data['Self_Employed'].mode()[0], inplace=True)
data['Credit_History'].fillna(data['Credit_History'].mode()[0], inplace=True)
data['Property_Area'].fillna(data['Property_Area'].mode()[0], inplace=True)

print("\nMissing Values After Handling:\n", data.isnull().sum())

label_encoder = LabelEncoder()

categorical_cols = ['Gender', 'Married', 'Dependents',
'Education', 'Self_Employed',
'Property_Area', 'Loan_Status']

for col in categorical_cols:
data[col] = label_encoder.fit_transform(data[col])
```

```
plt.figure()
sns.countplot(x='Loan_Status', data=data)
plt.title("Loan Approval Distribution")
plt.show()

plt.figure()
sns.heatmap(data.corr(), annot=False)
plt.title("Correlation Heatmap")
plt.show()

sample_input = np.array([[1, 1, 0, 1, 0, 5000, 2000,
150, 360, 1, 2]])

sample_input_scaled = scaler.transform(sample_input)

prediction = best_model.predict(sample_input_scaled)

if prediction[0] == 1:
print("\nLoan Status: APPROVED")
else:
print("\nLoan Status: REJECTED")

input_data = np.array([[Gender, Married, Dependents,
Education, Self_Employed,
ApplicantIncome, CoapplicantIncome,
LoanAmount, Loan_Amount_Term,
Credit_History, Property_Area]])

prediction = model.predict(input_data)

if prediction[0] == 1:
result = "Loan Approved ✅"
else:
result = "Loan Not Approved ❌"
```

```
SVCclassifier=SVC(kernel='rbf',max_iter=500)
SVCclassifier.fit(X_train, y_train)

y_pred=SVCclassifier.predict(X_test)

NBclassifier2 = GaussianNB()
NBclassifier2.fit(X_train,y_train)

y_pred=NBclassifier2.predict(X_test)

sample_input = np.array([[1, 1, 0, 1, 0, 5000, 2000,
150, 360, 1, 2]])

sample_input_scaled = scaler.transform(sample_input)

prediction = best_model.predict(sample_input_scaled)

if prediction[0] == 1:
print("\nLoan Status: APPROVED")
else:
print("\nLoan Status: REJECTED")
```

7. TESTING

7.1 TESTING DESCRIPTION:

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TEST:

UNIT TESTING

Unit testing involves the design of test cases that validate the internal logic is functioning properly, and that program input produces output. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of the individual unit before intergaration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit test perform basic test at component level and test the specific bussiness process, application, and/or system configuration. Unit test ensure that unique path of a business process performs accurately to the documented specification and contains clearly defined inputs and expected results.

INTEGRATION TESTING

Integration test are designed to test integrated software components to detemine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration test demonstrate that although the components were individually satifaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

VALIDATION TESTING

Every business application has got its own rules. We need to implement all the business rules in order to achieve a good output. We can enforce all the business rules

through the use of validation concepts. Maintaining security and integrity of a database is the most important factor in judging success of system.

In that project modules required field validator is used. The required field validator helps the corresponding textbox should not be empty. At the time the user easily understands the mistakes and helps to overcome the mistakes. Software validation was done to demonstrate conformity with requirement it satisfied all the requirements.

FUNCTIONAL TESTING:

Functional tests provide systematic demonstration that functions tested are available as specified by the business and technical requirements, system documentation and user manuals.

Functional testing is centered on the following items:

Valid Input	: identified classes of valid input must be accepted.
Invalid Input	: identified classes of invalid inputs must be rejected.
Functions	: identified functions must be exercised.
Output	: identified classes of application output must be exercised.
Systems/Procedures	: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

SYSTEM TESTING:

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

TEST STRATEGY AND APPROACH

Field testing will be performed manually and fuctional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the idenfified link.
- The entry screen, messages and responses must not be delayed.
- Features to be tested.
- Verify that the entries are of the correct format.
- No duplicate entries should be allowed.
- All links should take the user to the correct page.

Acceptance Testing:

User Acceptance testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results:

All the test cases mentioned above has been passed successfully. No defects encountered.

8. REPORTS

PERFORMANCE ANALYSIS:

Website performance optimization, the focal point of technologically superior website designs is the primary factor dictating success for Loan approval process. After all, unimpressive website performance kills admission process when the torture of waiting for slow Web pages to load frustrates visitors into seeking alternatives – impatience is a digital virtue! And also the ml algorithms used in our project will give the best accurate result to the user for Loan approval.

We created the following six chapter in-depth speed optimization guide to show you how important it is to have a fast loading, snappy website! Countless research papers and benchmarks prove that optimizing your sites' speed is one of the most affordable and highest ROI providing investments!

Lightning-fast page load speed amplifies visitor engagement, retention, and boosts sales. Instantaneous website response leads to higher conversion rates, and every 1 second delay in page load decreases customer satisfaction by 16 percent, page views by 11 percent and conversion rates by 7 percent according to a recent Aberdeen Group research.

DISCUSSION:

While discussions provide avenues for exploration and discovery, leading a discussion can be anxiety-producing: discussions are, by their nature, unpredictable, and require us as instructors to surrender a certain degree of control over the flow of information. Fortunately, careful planning can help us ensure that discussions are lively without being chaotic and exploratory without losing focus. When planning a discussion, it is helpful to consider not only cognitive, but also social/emotional, and physical factors that can either foster or inhibit the productive exchange of ideas.

Model	Accuracy
K Neighbors	91.111111
Random Forest	88.888889
SVM	86.666667
Decision Tree	84.444444
Logistic Regression	82.222222
Gradient Boost	75.555556
Gaussian NB	71.111111

9. CONCLUSION

The objective of this project is to predict the loan approval status of users. This online banking loan approval system reduces paperwork, minimizes wastage of bank resources, lowers manual effort, and saves valuable time for customers. By automating the approval process, the system ensures faster decision-making and improves the overall efficiency of bank operations. It also provides transparency to customers by offering immediate feedback on their loan applications.

The project followed a complete analytical workflow, starting from data cleaning and preprocessing, handling missing values, and performing exploratory data analysis, to building and evaluating machine learning models. Various algorithms were tested, and the model achieving the highest accuracy on the test dataset was selected for deployment. This application provides an efficient and reliable way to predict bank loan approvals, supporting both banks and customers in informed decision-making. In addition, the system can be easily extended to include additional features, such as risk scoring and automated recommendation, making it a scalable solution for modern banking needs.

FUTURE ENHANCEMENT

1. Integrate the bank loan approval prediction system with cloud platforms for real-time accessibility and scalability.
2. Enhance the system by incorporating advanced artificial intelligence techniques to improve prediction accuracy.
3. Implement automated risk assessment and scoring to provide detailed insights for loan approvals.
4. Expand the dataset with additional financial and demographic features to make the model more robust.
5. Develop a user-friendly mobile application to allow customers and bank officers to access predictions on-the-go.

10. REFERENCES

[1] Amruta S. Aphale , Dr. Sandeep R. Shinde, 2020, Predict Loan Approval in Banking System Machine Learning Approach for Cooperative Banks Loan Approval,InternationalJournalOfEngineeringResearch&Technology(IJERT) Volume 09, Issue 08 (August 2020)

[2] AshwiniS.Kadam,ShraddhaR.Nikam,AnkitaA.Aher,GayatriV.Shelke,Amar S.Chandgude, "Prediction for Loan Approval using Machine Learning Algorithm" (IRJET) Volume: 08 Issue: 04 | Apr 2021.

[3] M. A. Sheikh, A. K. Goel and T. Kumar, "An Approach for Prediction of Loan ApprovalusingMachineLearningAlgorithm,"2020InternationalConferenceon Electronics and Sustainable Communication Systems (ICESC), 2020, pp. 490- 494, doi: 10.1109/ICESC48915.2020.9155614.

[4] Rath,Golak&Das,Debasish&Acharya,Biswaranjan.(2021).ModernApproach for Loan Sanctioning in Banks Using Machine Learning. Pages={179-188} 10.1007/978-981-15-5243-4_15.

[5] Vincenzo Moscato, Antonio Picariello, Giancarlo Sperlí, A benchmark of machine learningapproachesforcreditscoreprediction,ExpertSystemswithApplications, Volume 165, 2021, 113986, ISSN 0957-4174.

[6] YashDivate,PrashantRana,PratikChavan,"LoanApprovalPredictionUsing Machine Learning" International Research Journal of Engineering and Technology (IRJET) Volume: 08 Issue: 05 | May 2021.

[7] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., New York, NY, USA: Springer, 2009.

[8] C. M. Bishop, *Pattern Recognition and Machine Learning*, New York, NY, USA: Springer, 2006.

[9] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*, Cambridge, MA, USA: MIT Press, 2012.

[10] S. Raschka and V. Mirjalili, *Python Machine Learning*, 3rd ed., Birmingham, U.K.: Packt Publishing, 2019.

[11] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 2nd ed., Sebastopol, CA, USA: O'Reilly Media, 2019.

[12] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[13] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[14] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[15] I. H. Witten, E. Frank, and M. A. Hall, *Data Mining: Practical Machine Learning Tools and Techniques*, 3rd ed., Burlington, MA, USA: Morgan Kaufmann, 2011.

[16] M. Kuhn and K. Johnson, *Applied Predictive Modeling*, New York, NY, USA: Springer, 2013.

9.2 WEBLIOGRAPHY:

[1] Scikit-learn Official Documentation. Available: https://scikit-learn.org/stable/

[2] Pandas Documentation. Available: https://pandas.pydata.org/docs/

[3] NumPy Documentation. Available: https://numpy.org/doc/

[4] Flask Official Documentation. Available: https://flask.palletsprojects.com/

[5] Kaggle – Loan Prediction Dataset. Available: https://www.kaggle.com/

[6] Towards Data Science – Loan Prediction using Machine Learning. Available: https://towardsdatascience.com/